

Proximal Policy Optimization Algorithms 2017

梗概

PPO (Proximal Policy Optimization) 的提出是因为传统的 policy gradient 算法如 TRPO 采样一次，更新一次然后数据就被丢弃，不仅计算复杂，而且数据利用率仍不够高。因此作者提出了 PPO family，保持了 TRPO 的 Trust Region 的思想，提出了 **clipped probability ratio** 等技巧达到和 TRPO 相似甚至更优的效果。

背景

当时主流的强化学习算法主要有三个流派，作者进行了评价

- **Deep Q Learning**

fails on many simple problems

特别是在连续动作空间

- **Vanilla Policy Gradient**

poor data efficiency

太浪费数据

- **Trust Region**

relatively complicated

效果很好，但是太复杂

attain the data efficiency and reliable performance of TRPO while using only first-order optimization

作者希望保留 TRPO 的数据效率和稳定性，但只使用一阶优化

(如果你对 TRPO 的数学理论感兴趣，可以移步本人的 Trust Region Policy Optimization 篇，对你理解后面这些数学公式可能有所帮助)

Policy Gradient

PG 的核心就是我们要更新神经网络的参数

$$\theta + \alpha g \rightarrow \theta$$

而这个 g 就是

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (1)$$

中间的 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 表示如何增加当前动作概率

- $\hat{A}_t$ (advantage) 表示当前动作好不好
- π_θ 表示策略
- a_t 表示动作
- s_t 表示状态

g 则是通过微分

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right] \quad (2)$$

得到的。作者称 **公式 (2)** 为 objective function。但是其实这不是强化学习真正的优化目标。

强化学习最原始的目标是

$$J(\theta) = E_{\tau \sim \pi_\theta} \left[\sum_t r_t \right]$$

即 **期望累计奖励**。

但 $J(\theta)$ 太难求，好在 Williams 给出了它的梯度表示（详细推导请移步 Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning 的论文综述帖）

$$\nabla_\theta J(\theta) = E [\nabla_\theta \log \pi_\theta(a|s) A(s, a)]$$

这就是 **公式 (1)**

但是有个问题，我们只有梯度表示，没有原本的 Loss function 怎么办？

作者说 “Gradient is all you need”（玩梗），我们只需要正确的坡度就行。

这就好比爬山，我们想要尽可能路径短的爬到山顶，那么我们只需要考虑每个位置的坡度就好，无需考虑山在哪，什么朝向，为了研究这个最短路程，我们甚至可以在自家后院重新构建一个坡度处处与原山相等的假山，因为我们只需要的是路怎么走。

同样的思想，我们只需要构造一个“假山” $L(\theta)$ 有

$$\nabla L(\theta) = \nabla J(\theta)$$

这个 L 在强化学习中就是 **代理目标 surrogate objective**

如何构造出让梯度好算，训练稳定，数据利用率高的代理目标就成了我们需要考虑的问题

Trust Region Methods

在 TRPO 的文章中给出了优化问题的近似代理目标

$$\begin{aligned} \max_{\theta} \quad & \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \\ \text{subject to} \quad & \hat{\mathbb{E}}_t [KL(\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t))] \leq \delta. \end{aligned}$$

Trust Region 实际上的意思就是每次更新策略的时候不能改太多

例如在 $A_t > 0$ 的时候为了最大化优化目标，显然是把 $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 拉满到无穷，但是这样显然不合理

这有点像股票交易中的仓位管理。

当一只股票持续上涨时，我们当然希望继续加仓，但很少有人会因为一次判断正确就直接 all-in。因为市场具有不确定性，即便当前趋势向上，过于激进的操作仍可能导致巨大的回撤。

更常见的做法是逐步调整仓位：上涨时逐渐加仓，下跌时逐渐减仓，并在每次调整后重新评估市场情况，而不是一次性完成所有决策。

KL 约束就是我们实现这种策略变化约束的方案: **让每一次策略的调整小于一个阈值**

但是问题就出在这，TRPO 进行了三次近似并需要进行共轭梯度，Fisher Matrix 等复杂数学计算

作者认为，移除这些二阶优化的关键在于不应该将 KL 写成 Constraint Form 而是更适合 Penalty Form

$$\max E[r_t A_t] - \beta KL$$

但是作者发现仅这样变换形态并不能完全解决问题，还是要对策略更新幅度做出限制，这次不用限制 KL I，而是直接限制 r_t 。这就是作者提出的 **clipped** 方法

Clipped Surrogate Objective

定义 $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ ，TRPO 需要最大化的目标就表示为

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] = \hat{\mathbb{E}}_t [r_t(\theta) \hat{A}_t]. \quad (3)$$

为了乘法哪些让 r_t 偏离 1 ($r_t(\theta_{\text{old}}) = 1$) 的更新作者进行了 **clip**

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]. \quad (4)$$

丢弃了 KL 项，直接使用非常简单且暴力的方式限制更新范围

Adaptive KL Penalty Coefficient

这一板块实际上讨论为什么最后选择 Clip 而不是 KL Penalty

这就是我们刚刚讨论的 KL 的 Penalty Form

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta KL(\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t)) \right] \quad (5)$$

$$d = \hat{\mathbb{E}}_t [KL(\pi_{\theta_{old}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t))]$$

adaptive的意思就是 β 也自适应进行更新

$$\begin{cases} \beta \leftarrow \beta/2, & d < d_{\text{targ}}/1.5, \\ \beta \leftarrow 2\beta, & d > 1.5 d_{\text{targ}}. \end{cases}$$

作者解释这个方案被放弃还是因为

- 仍存在对 β 的调参问题
- 效果任不如 clip

算法实现

实现 PPO 实际上只需要将原来的 Policy Gradient Loss 替换为 PPO Loss

PPO Loss 本质上继承了 TRPO 和 Actor-Critic 的思想

PPO 在训练时，Advantage 需要估计

$$A(s, a) = Q(s, a) - V(s)$$

所以必须训练 $V(s)$

为了综合考虑 Policy, Value function 和 Trust Region, 作者定义了

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t [L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_{\theta}](s_t)]. \quad (6)$$

- L^{CLIP} 策略优化
- L^{VF} value function loss, 一般就是简单的 MSE $(V_{\theta}(s_t) - V_t^{\text{target}})^2$ 训练 Critic
- $S[\pi]$ Entropy Bonus, 鼓励探索, 防止策略过早确定

根据 **公式 (5)** L^{CLIP} 中有 $\hat{A}_t$ 需要计算, 作者希望能像 A3C, A2C 那样, 跑完 T 步之后进行一次更新

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V(s_T). \quad (7)$$

可以拆开两部分看

- $r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1} r_{T-1}$
从 t 开始直接到截断点 T 能看到所有的奖励
- $\gamma^{T-t} V(s_T)$
这是 Bootstrap, 后面虽然看不到, 但是让 Value Network 猜

面对传统的 A3C, A2C 式的 A 值计算，作者想要将其拓展，作者引入了 **GAE** Generalized Advantage Estimation 广义优势估计

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \quad \text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (8)$$

δ_t 实际上就是 **TD Error**，大于 0 时表示实际结果比 Value 预期好，反之则差

作者将 A 表示成了多个 TD Error 加权求和的形式，**公式 (7)** 就变成了 **公式 (8)** 在 $\lambda = 1$ 时的特殊形式

- 当 $\lambda = 1$ 时，接近 Monte Carlo，偏差小方差大
- 当 $\lambda = 0$ 时， $A_t = \delta_t$ 偏差大方差小

这实际上是在进行 Monte Carlo 和 TD 之间的 **Bias-Variance Tradeoff**

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1, 2, ... do
  for actor=1, 2, ..., N do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for
```

实验总结

实验对多种 surrogate objectives 进行了分析

- No clipping or penalty:

$$L_t(\theta) = r_t(\theta)\hat{A}_t$$

- Clipping:

$$L_t(\theta) = \min \left(r_t(\theta)\hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right)$$

- KL penalty (fixed or adaptive):

$$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta KL(\pi_{\theta_{\text{old}}}, \pi_{\theta})$$

实验发现

1. vanilla policy gradient 不稳定

普通 Policy Gradient 在重复利用同一批数据时容易出现

- 策略更新过大
- KL 快速增长
- 性能剧烈波动

说明仅依赖梯度方向不足以保证训练稳定性

2. Adaptive KL Penalty 有效但不稳定

适应 KL Penalty 能够一定程度控制策略变化

- β 调节存在滞后
- KL 波动较大
- 对超参数敏感

3. PPO-Clip 表现最佳

Continuous Control 任务中比较了 TRPO, PPO, Vanilla PG 在 **连续动作空间** 中的表现

结果表明

1. PPO 达到和 TRPO 相当的性能

大多数任务上:

- PPO 收敛速度接近 TRPO
- 最终奖励接近或超过 TRPO

2. PPO 样本利用率更高

3. PPO 计算开销显著降低

在 Atari Games 上对 **离散动作任务** 进行实验比较, 验证 PPO 是否能从连续控制推广到离散动作场景

实验结果表明

1. PPO 同样适用于离散动作空间

2. PPO 超过 A2C/A3C

3. PPO 保持良好的泛化能力

未来工作

作者认为 PPO 可能也适用于

- 更大的神经网络
- 更复杂的强化学习任务
- 更宽广的策略优化问题